课程介绍

- 逻辑回归
  - 分类算法
- 决策树
  - 分类算法

# 逻辑回归介绍

逻辑回归（Logistic Regression）是机器学习中的**一种分类模型**，逻辑回归是一种分类算法。

## 逻辑回归的应用场景

二分类利器

## 逻辑回归的原理

线性回归输出是逻辑回归的输入

### 激活函数

- sigmoid函数
  - 线性回归输出是一个数值，通过sigmoid转化成一个概率[0,1]
- 二分类问题
  - 阈值　默认0.5
  - 正例 > 阈值 == > 正例
  - 负例 < 阈值 == > 负例

## 损失以及优化

- y = 1 时，希望逻辑输出结果越大越好
- y = 0 时，希望逻辑输出结果越小越好

### 优化

**提升原本属于1类别的概率，降低原本是0类别的概率。**

## 逻辑回归api介绍

`sklearn.linear_model.LogisticRegression(solver='liblinear', penalty='l2', C = 1.0)`

参数：

- solver可选参数:{'liblinear', 'sag', 'saga','newton-cg', 'lbfgs'},
  - 默认: 'liblinear'；用于优化问题的算法。
  - 对于小数据集来说，“liblinear”是个不错的选择，而“sag”和'saga'对于大型数据集会更快。
  - 对于多类问题，只有'newton-cg'， 'sag'， 'saga'和'lbfgs'可以处理多项损失; “liblinear”仅限于“one-versus-rest”分类。
- penalty：正则化的种类
- C：正则化力度

## 案例：癌症分类预测-良 / 恶性乳腺癌肿瘤预测

建模分析

1. 获取数据
2. 数据基本处理
   - 缺省值处理
   - 确定特征值目标值
   - 分割数据
3. 特征工程
   - 数据标准化
4. 建立模型
   - 逻辑回归
5. 模型评估

# 分类评估方法

1. 混淆矩阵
2. 精确率
3. 召回率
4. roc曲线、auc指标

## 精确率与召回率

### 混淆矩阵

预测结果(Predicted Condition)与正确标记(True Condition)之间存在四种不同的组合，构成混淆矩阵(适用于多分类)

| | | 预测结果 | |
|---|---|---|---|
| | | 正例 | 假例 |
| 真实结果 | 正例 | 真正例TP | 伪反例FN |
| | 假例 | 伪正例FP | 真反例TN |

### 精确率(Precision)与召回率(Recall)

- 精确率：预测结果为正例样本中真实为正例的比例

  `精确率 = TP /（TP + FP） 查的准`

- 召回率：真实为正例的样本中预测结果为正例的比例

  `召回率 = TP /（TP + FN） 查得全，对正样本的区分能力`

  `准确率 = （TP + TN） / （TP + TN + FN + FP）`

### F1-score

F1-score，反映了模型的稳健型

### 分类评估报告api

`sklearn.metrics.classification_report(y_true, y_pred, labels=[], target_names=None )`

- y_true：真实目标值
- y_pred：估计器预测目标值
- labels:指定类别对应的数字
- target_names：目标类别名称
- return：每个类别精确率与召回率

## ROC曲线与AUC指标

- ROC：选择不同的阈值计算FPR和TPR落在二维平面上
  - x: FPR 越小越好
  - y: TPR越大越好

### 2.3 AUC指标

AUC的概率意义是随机取一对正负样本，正样本得分大于负样本得分的概率

AUC的范围在[0, 1]之间，并且越接近1越好，越接近0.5属于乱猜

- **AUC=1，完美分类器**
- **0.5<AUC<1，优于随机猜测。这个分类器（模型）妥善设定阈值的话，能有预测价值。**

### AUC计算API

`from sklearn.metrics import roc_auc_score`

- sklearn.metrics.roc_auc_score(y_true, y_score)
  - 计算ROC曲线面积，即AUC值
  - y_true：每个样本的真实类别，必须为0(反例),1(正例)标记
  - y_score：预测得分，可以是正类的估计概率、置信值或者分类器方法的返回值

## ROC曲线得绘制

选择不同的阈值,计算TPR和FPR落在二维平面

## 决策树算法

分类算法

### 决策树算法简介

决策树:

1. 树形结构
2. 每个内部节点代表一个判断
3. 每个叶子节点代表一个分类结果

**本质是一颗由多个判断节点组成的树**

构建树:

1. 如何判断特征值先后顺序
2. 每个特征里面切割点划分

# 4.2 决策树分类原理

1. ID3--信息增益
2. C4.5--信息增益率
3. CART--基尼指数

## 1 熵信息熵（Entropy)

- 数据混乱程度的度量单位
  - 有序: 熵小
  - 混乱:熵大
- 信息熵的计算:

$$Ent(D) = -\sum_{k=1}^{n} \frac{C^k}{D} log \frac{C^k}{D}$$

## 决策树的划分依据一----信息增益(ID3)

信息增益: 通过某特征值划分决策前后的熵值差

**信息增益 = entroy(前) - entroy(后)**

entroy(后)==条件熵

- 条件熵的计算:

$$Ent(D,a) = \sum_{v=1}^{V} \frac{D^v}{D} Ent(D^v) = -\sum_{v=1}^{V} \frac{D^v}{D} \sum_{k=1}^{K} \frac{C^{kv}}{D_v} log \frac{C^{kv}}{D_v}$$

选择信息增益大的属性进行第一决策依据

## 决策树的划分依据二----信息增益率(C4.5)

信息增益优先选择类别多的特征作为第一决策依据

选择信息增益率大的作为第一决策依据

$$Gain_ratio(D,a) = \frac{Gain(D,a)}{IV(a)}$$

其中

$$IV(a) = -\sum_{v=1}^{V} \frac{D^v}{D} log \frac{D^v}{D}$$

while（不纯）

1. 计算熵值
2. 计算条件熵值
3. 计算信息增益
4. 计算信息增益率
5. 选择信息增益率最大进行条件切分

### 为什么使用C4.5要好

1. **用信息增益率来选择属性**
   - 克服了用信息增益来选择属性时偏向选择值多的属性的不足。
   - 信息增益的优化
2. **采用了一种后剪枝方法**
   - 避免树的高度无节制的增长，避免过度拟合数据
3. **对于缺失值的处理**

## 决策树的划分依据三 ----基尼值和基尼指数(CART)

一定是二叉树

### 概念

**基尼值Gini（D）： 故，Gini（D）值越小，数据集D的纯度越高。**

$$Gini(D) = \sum_{k=1}^{|y|} \sum_{k^{'} \neq k} p_k p_{k^{'}} = 1 - \sum_{k=1}^{|y|} p_k^2$$

基尼指数Gini——index（D**----CART决策树

$$Gini_index(D, a) = \sum_{v=1}^{V} \frac{D^v}{D} Gini(D^v)$$

CART的算法流程

while(节点不纯)

1. 遍历每个变量的每一种分割方式，找到最好的分割点
2. 分割成两个节点N1和N2
3. 每个节点足够“纯”为止

# 小结

## 常见决策树的启发函数比较

### ID3算法

- 存在的缺点：
  1. 偏向类别多的属性
  2. 只能构建离散

### C4.5算法

**做出的改进(为什么使用C4.5要好)**

1. 用信息增益率来选择属性
2. 可以处理连续数值型属性
3. 采用了一种后剪枝方法
4. 对于缺失值的处理

**C4.5算法的优缺点**

- 优点：
  - 产生的分类规则易于理解，准确率较高
- 缺点
  - 算法低效
  - C4.5只适合于能够驻留于内存的数据集

### CART算法

二叉树模型

基尼指数值

- `**C4.5不一定是二叉树，但CART一定是二叉树**`

#### 多变量决策树(multi-variate decision tree)

**分类决策不应该是由某一个特征决定的，而是应该由一组特征决定的**

### 决策树变量的两种类型

1. 名称型(离散型)
2. 数字型（Numeric）比如年收入

### 如何评估分割点的好坏？

使得每一类都很“纯”

# cart剪枝

1. 预剪枝
2. 后剪枝

## 为什么要剪枝

- 随着节点数的增加决策树容易过拟合
- **出现这种情况的原因：**
  1. 噪声、样本冲突，即错误的样本数据。
  2. 特征即属性不能完全作为分类标准。
  3. 巧合的规律性，数据量不够大。

## 常见的剪枝方法

### 预剪枝

1. 指定每个节点最小样本数
2. 指定高度或者深度
3. 指定节点熵小于某个阈值

### 后剪枝

在已生成过拟合决策树上进行剪枝，可以得到简化版的剪枝决策树。

# 特征工程-特征提取

## 特征提取

- **将任意数据（如文本或图像）转换为可用于机器学习的数字特征**
- 特征提取类别:
  1. 字典特征提取
  2. 文本特征提取
  3. 图像特征提取

## 特征提取API

`sklearn.feature_extraction`

### 字典特征提取

`sklearn.feature_extraction.DictVectorizer(sparse=True,…)`

- DictVectorizer.fit_transform(X)
  - X:字典或者包含字典的迭代器返回值

    - 返回sparse矩阵
- DictVectorizer.get_feature_names() 返回类别名称
- sparse矩阵

  对应的位置 与值组成

**流程分析**

1. 构建数据
2. 实例化api
3. 执行转化

### 文本特征提取

`sklearn.feature_extraction.text.CountVectorizer(stop_words=[])`

- 返回词频矩阵
- CountVectorizer.fit_transform(X)
    - X:文本或者包含文本字符串的可迭代对象
    - 返回值:返回sparse矩阵
- CountVectorizer.get_feature_names() 返回值:单词列表
- stop_words 停用词表

  文本频数统计

**流程分析**

1. 实例化api
2. 执行转换

### jieba分词处理